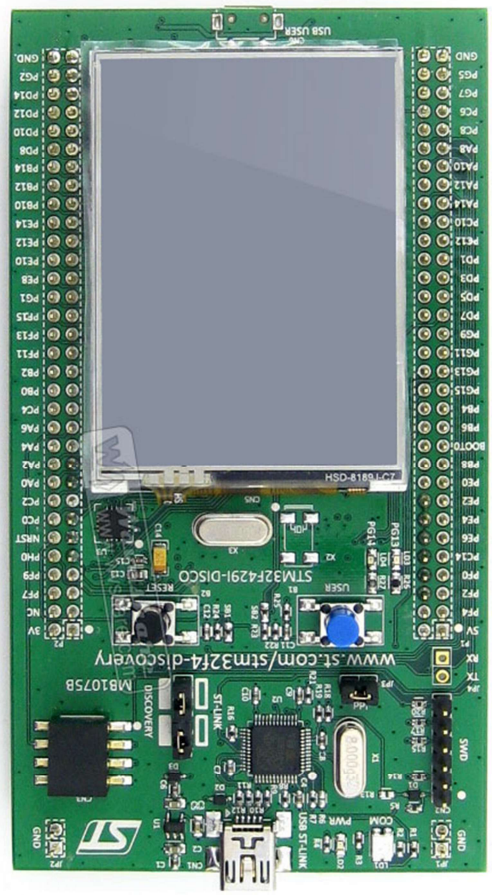
2024-2025

Compte rendu du projet : STYLO VOLTMETRE NUMERIQUE A AFFICHAGE DEPORTE



Boubacar DIALLO

Oussama HARBALI

2024-2025

Table des matières

Introduction 2
Présentation du projet 2
Câblage entre le MCP et la carte STM32 3
Présentation des composants 3
Développement 6
Configuration du STM32CubeMX 6
Programmation sur STM32CudeIDE 6
Difficultés et solutions proposées 10
Problèmes avec la Configuration Automatique du SPI 10
Optimisation et Stabilité de l'Affichage 11
Conclusion 11

Introduction

Dans le cadre de notre SAE3 en deuxième année de BUT GEII, nous avons travaillé en groupe sur un projet dont l'objectif principal vise à faire dialoguer un système électronique simple et efficace, capable de récupérer une information, de la traiter et de l'afficher de manière claire et intuitive. En appuyant sur un bouton, le dispositif mesure une tension électrique, puis transmet cette information à un microcontrôleur qui l'interprète et l'affiche sur un écran. L'objectif est de se concentrer sur le traitement et l'affichage des données via une carte STM32, en garantissant une expérience utilisateur fluide et précise.

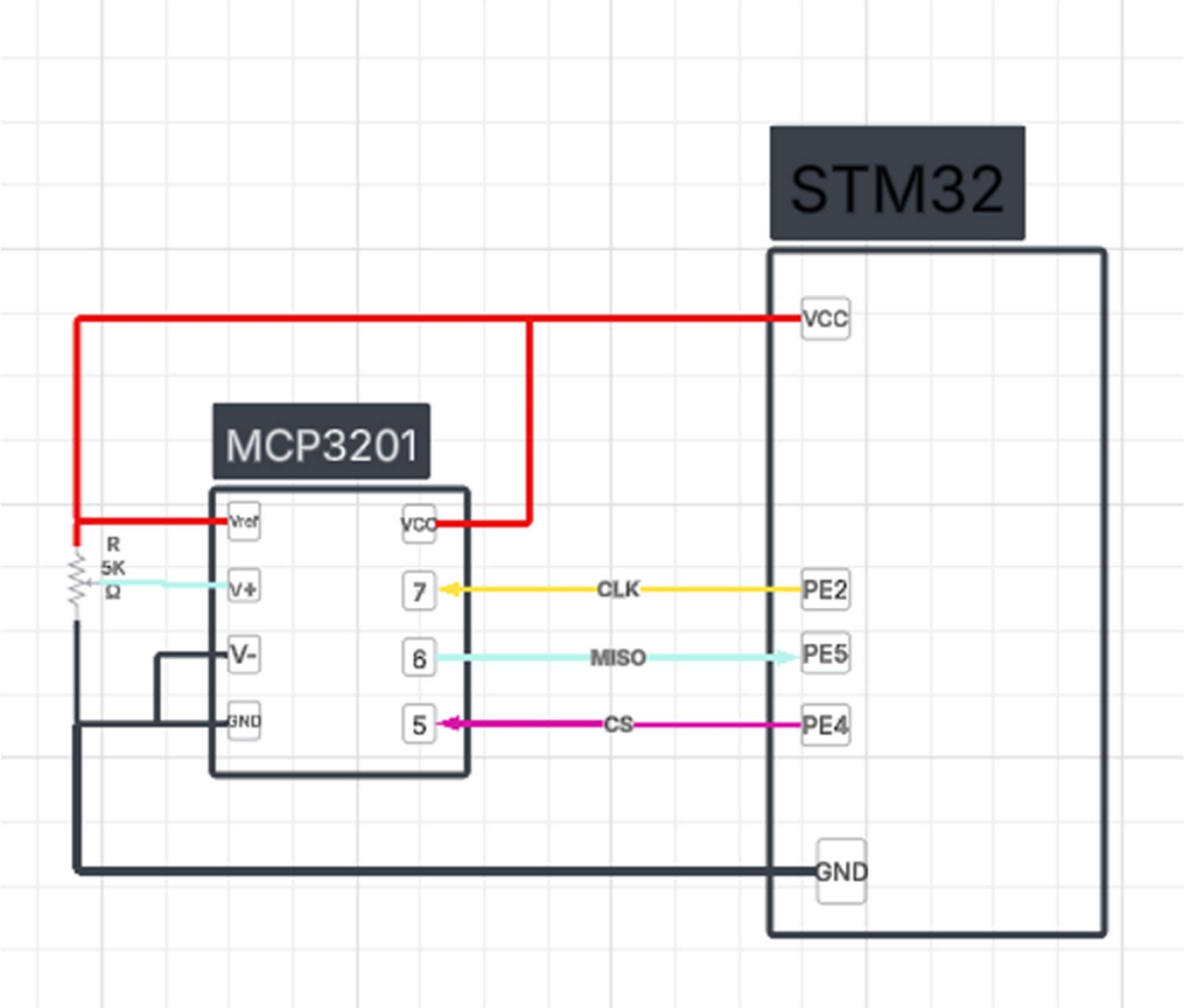
Présentation du projet

Au cœur du système se trouve le MCP3201, un composant essentiel chargé de mesurer une tension électrique et de la convertir en données numériques. Ces données sont ensuite transmises au microcontrôleur via une communication rapide et fiable. Le microcontrôleur, quant à lui, agit comme le cerveau du système : il reçoit les informations, les traite, et les affiche de manière lisible sur un écran. Ainsi, l'utilisateur peut visualiser en temps réel la tension mesurée, sans nécessiter de connaissances techniques approfondies.

Le projet présentera donc les caractéristiques suivantes :

- Gamme de tension de mesure 0-14V
- Alimentation par pile incorporée au stylo
- Déclenchement par appui sur un bouton sur le stylo
- Précision de mesure 50mV
- Impédance d'entrée > 1M ohm
- Temps entre 2 mesures : 1seconde
- Liaison Stylo-Affichage sur un seul fil en vue de préparer une liaison sans fil
- Affichage réalisé à partir d'une carte d'évaluation STM32F429 discovery avec écran

Câblage entre le MCP et la carte STM32

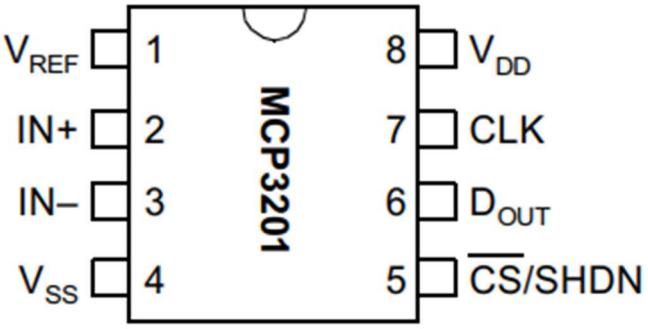


Ce projet constitue une excellente occasion de mettre en pratique nos compétences en électronique et en programmation, tout en prenant en compte l'ergonomie et la simplicité d'utilisation. Il s'agit de transformer une mesure de tension, souvent perçue comme abstraite, en une information facilement compréhensible pour l'utilisateur.

Ce travail nous a permis de développer et de renforcer les quatre compétences fondamentales du BUT GEII. Nous allons à présent détailler notre progression en fonction de ces compétences.

Présentation des composants

Le **MCP3201** est un convertisseur analogique-numérique (CAN) de **12 bits**, conçu pour mesurer une tension différentielle entre ses broches **IN+** et **IN-**, puis la convertir en une valeur numérique transmise à un microcontrôleur via une interface **SPI**.



Son fonctionnement repose sur trois étapes principales :

Acquisition de la tension

Lorsqu'une mesure est demandée, le MCP3201 capture la différence de potentiel entre ses entrées **IN+** et **IN-**. Cette tension analogique est alors convertie en une valeur numérique sur **12 bits**, représentant avec précision l'amplitude du signal mesuré.

$$\text{Digital Output Code} = \frac{4096 * V_{IN}}{V_{REF}}$$

Where:

V_{IN} = Analog Input Voltage = $V(_{IN}+) - V(_{IN}-)$

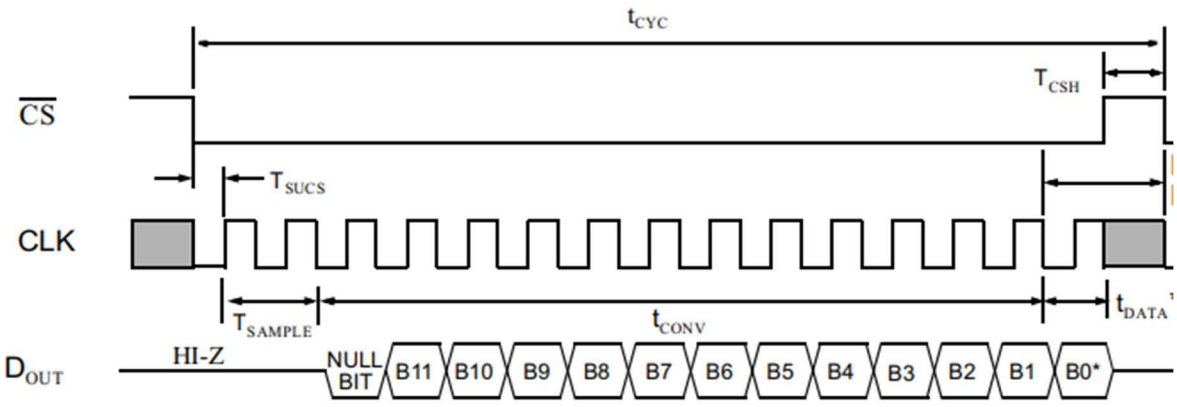
V_{REF} = Reference Voltage

Communication avec la STM32

La STM32 initie l'échange en activant le **signal CS (Chip Select)**. Une fois la communication SPI engagée, le MCP3201 transmet ses données via la ligne **MISO**, synchronisées avec l'horloge SPI générée par la STM32. À la réception, le microcontrôleur extrait les **12 bits de mesure**, les interprète et les affiche sous une forme exploitable, permettant ainsi une lecture claire et précise de la tension mesurée.

Transmission des données

Pour communiquer avec la STM32, le MCP3201 envoie une séquence de **15 bits** au total. Les **deux premiers bits** correspondent à la phase d'échantillonnage, suivis d'un **bit NULL** sans signification, puis des **12 bits** de données utiles représentant la tension mesurée. La transmission s'effectue en **MSB First**, c'est-à-dire en commençant par le bit de poids fort.



STM32F429IDIS

La **STM32F429IDIS** est une carte de développement basée sur le microcontrôleur **STM32F4** de STMicroelectronics. Ce microcontrôleur est reconnu pour sa puissance, sa polyvalence et sa capacité à gérer des tâches complexes, ce qui le rend idéal pour une variété d'applications, notamment dans le domaine des systèmes embarqués.

Dans le cadre de ce projet, la STM32F429IDIS sera utilisée pour communiquer avec le **MCP3201**, un convertisseur analogique-numérique (CAN), via le protocole **SPI** (en l'occurrence, **SPI4**). Le microcontrôleur jouera un rôle central dans la récupération, le traitement et l'affichage des mesures de tension fournies par le MCP3201.

Fonctionnement du système :

Acquisition des données :
La STM32F429IDIS reçoit les données numériques du MCP3201 via l'interface **SPI4**. Ces données représentent les mesures analogiques converties en valeurs numériques.

Traitement des données :
Une fois les données récupérées, elles sont traitées pour être converties en valeurs de tension exploitables. Ce traitement inclut des calculs pour interpréter les valeurs brutes du CAN en tensions mesurables.

Affichage des résultats :
Les valeurs de tension calculées sont ensuite affichées en temps réel sur l'écran intégré à la carte de développement, permettant une visualisation immédiate des mesures.

Fonctions utilisées :
Pour gérer les entrées/sorties et la communication avec le MCP3201, les fonctions suivantes de la bibliothèque **HAL** (Hardware Abstraction Layer) seront utilisées :

- **HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState)** : Pour contrôler l'état des broches GPIO (par exemple, pour activer/désactiver le CS).
- **HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)** : Pour lire l'état des broches GPIO (par exemple, pour lire le MCP).

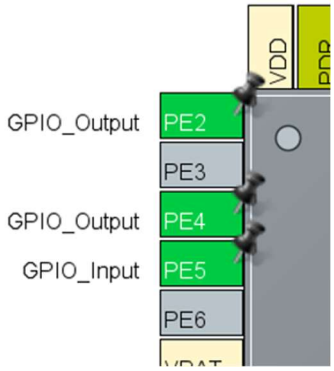
Développement

Configuration du STM32CubeMX

Dans le cadre de cette SAE, notre objectif est de mettre en place une communication entre le MCP3201 et la carte STM32 en utilisant l'interface SPI4 Afin de mieux comprendre le fonctionnement du protocole SPI, nous avons opté pour une configuration manuelle plutôt que l'activation automatique de SPI4 dans STM32CubeMX.

Pour cela, nous avons défini et contrôlé les broches essentielles via le code :

- **PE4** configurée en sortie,
- **PE5** configurée en entrée,
- **PE2** configurée en sortie.

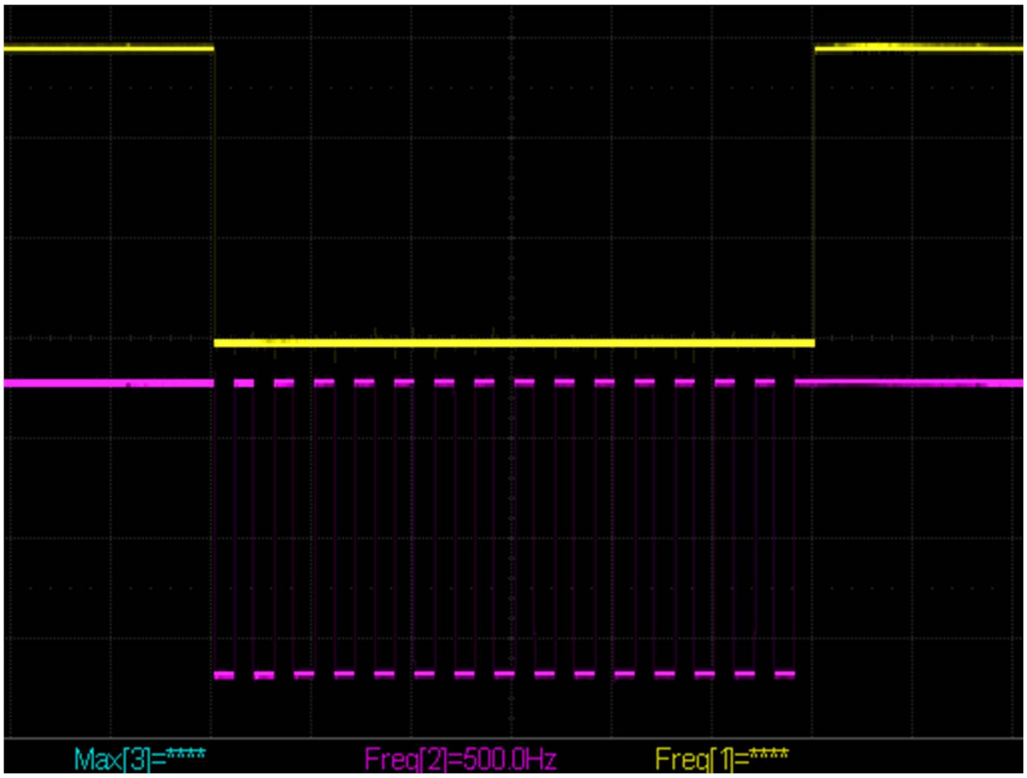


Une fois cette configuration réalisée, nous avons généré le code et procédé à la compilation, qui s'est déroulée avec succès. La suite de l'implémentation a été réalisée en suivant la méthodologie du TP1.

Programmation sur STM32CudeIDE

Générer le CS et le signal d'horloge

Nous avons commencé par générer le signal **CS** (Chip Select) ainsi que le signal d'horloge. Étant donné que la configuration est réalisée manuellement, nous avons défini la fréquence d'horloge à **500 kHz**. Après implémentation, nous avons obtenu le signal illustré ci-dessus, confirmant la bonne génération du signal d'horloge et du CS.

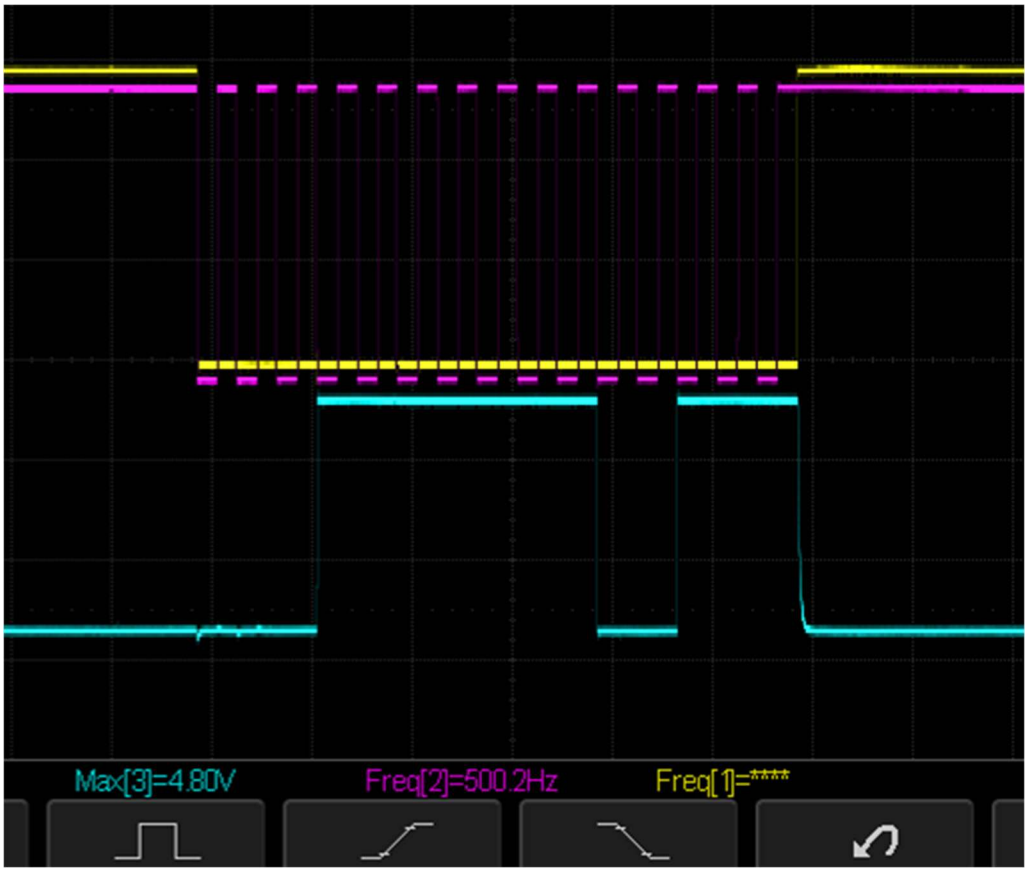


Lire le MCP et traitement des données

Pour la lecture des données, le CS (Chip Select), relié à PE4, est mis à l'état bas pour signaler au MCP3201 que la communication va commencer. Une boucle for s'exécute 15 fois afin de générer 15 impulsions d'horloge et de récupérer les 15 bits de données transmis par le MCP3201. À chaque itération, l'horloge (SCK, PE2) est mise à l'état bas, initiant ainsi un nouveau cycle d'horloge. La variable donne_numerique est décalée à gauche (<< 1) pour faire de la place au prochain bit.

En utilisant la fonction HAL_GPIO_ReadPin(GPIOx, GPIO_Pin), on lit l'état de la broche MISO (PE5). Le bit reçu est ensuite ajouté à donne_numerique. Si la broche PE5 est à l'état haut (1), on ajoute un 1 au LSB de donne_numerique (donne_numerique |= 1) ; sinon, la valeur reste inchangée (le bit ajouté est 0). L'horloge (SCK) est ensuite remise à l'état haut, validant ainsi la transmission du bit. Une fois les 15 bits lus, le CS est remis à l'état haut pour signaler la fin de la communication avec le MCP3201.

On a donc réussi à envoyer 15 impulsions d'horloge au MCP3201 et à récupérer les 15 bits de données issus de la conversion analogique-numérique. Ces données sont stockées dans la variable donne_numerique, prêtes à être traitées par la suite. Ce processus nous a permis d'obtenir les oscillogrammes suivants, illustrant la transmission des données entre le microcontrôleur et le MCP3201. On observe clairement les impulsions d'horloge ainsi que l'évolution des signaux échangés, confirmant le bon déroulement de la communication et l'acquisition des données numériques issues de la conversion.



Affichage et Calcul de la Tension Mesurée

Nous allons maintenant **convertir une valeur numérique issue d'un convertisseur analogique-numérique (CAN) en une tension électrique**, puis d'afficher cette tension sur un écran LCD. Une barre de progression est également mise à jour pour représenter graphiquement l'évolution de la mesure.

La mesure commence par la lecture d'une valeur numérique fournie par le **CAN**, qui convertit une tension analogique en un nombre entier. Ce nombre est ensuite transformé en tension à l'aide de la formule :

$$V = \frac{\text{donne_numerique} * 4.61}{4096}$$

où :

- donne_numerique est la valeur obtenue du CAN (résolution 12 bits, soit 4096 pas).
- 4.61 est le facteur d'échelle correspondant à la tension maximale fournit par la STM32.
- 4096 est le nombre total de valeurs possibles pour une conversion sur 12 bits.

Un **facteur de multiplication de 2.8** est ensuite appliqué pour obtenir une seconde tension mise à l'échelle :

V_{mulit}= V *2.8

Ce facteur est utilisé pour transformer une plage de **0 à 5 V** en **0 à 15 V**, conformément aux spécifications du cahier des charges. Les tensions calculées sont converties en **chaîne de caractères** pour être affichées de manière lisible sur l'écran LCD. La tension est initialement un nombre à virgule flottante (exemple : **3.456 V**). Pour un affichage plus lisible, elle est formatée sous la forme X, YY, où **X** est la partie entière ; **YY** représente les **deux premiers chiffres après la virgule**, exprimés en **centièmes de volts**.

Pour cela, le programme effectue les opérations suivantes et Ensuite, la valeur est transformée en **texte** grâce à la fonction sprintf() :

```
uint16_t partie_entiere = (uint16_t)voltage; // Partie entière
float partie_decimale_float = voltage - partie_entiere; // Partie décimale
sprintf(buffer_voltage, "%d,%d V", partie_entiere, (int)(partie_decimale_float * 100));
```

%d est utilisé pour afficher des nombres entiers.
partie_entiere est insérée directement dans la chaîne.
partie_decimale_float * 100 permet de convertir la partie décimale en **centièmes de volts** pour un affichage précis.

Une fois les valeurs formatées, elles sont affichées sur l'écran LCD :

- buffer_voltage est affiché à la ligne 11.
- buffer_voltage_multiplied est affiché à la ligne 8.
- La couleur de fond est définie avec BSP_LCD_SetBackColor(LCD_COLOR_LIGHTBLUE).
- La couleur du texte est mise en noir avec BSP_LCD_SetTextColor(LCD_COLOR_BLACK).

Nous avons ajouté une barre de progression rouge qui représente visuellement la tension mesurée par rapport à 14V. Une fonction DrawPercentageBar() est appelée avec la valeur voltage_multiplied. Cette fonction met à jour une barre de progression qui représente graphiquement le niveau de la tension mesurée.

- On évite les dépassements en bloquant la barre à 100%.
- Seule la partie qui change est rafraîchie, évitant ainsi les scintillements.

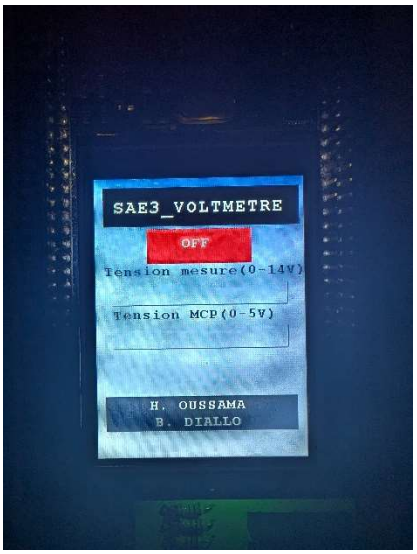
Le bouton est un rectangle affiché en haut de l'écran, juste sous le titre. Il change dynamiquement :

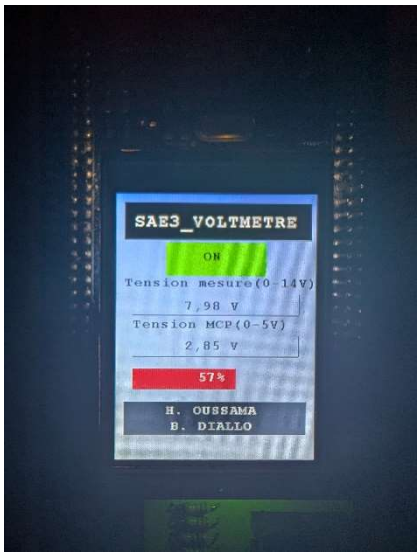
- Si la mesure est active, le bouton affiche "STOP" en rouge.
- Si la mesure est arrêtée, le bouton affiche "START" en vert.

Le bouton est dessiné avec la fonction Drawbutton() avec un Affichage dynamique qui permet que le texte et la couleur changent automatiquement.

Un simple toucher sur l'écran permet d'inverser l'état du bouton et de la mesure avec la fonction CheckTouch() :

- L'écran tactile est scanné en continu.
- Un appui active/désactive la mesure en inversant measuring_enabled.
- Le bouton change dynamiquement entre "START" et "STOP"





Difficultés et solutions proposées

Problèmes avec la Configuration Automatique du SPI

La communication SPI (Serial Peripheral Interface) est essentielle pour l'échange de données entre le STM32 et le **convertisseur analogique-numérique (MCP3201)**. Au départ, on a utilisé la **configuration automatique de STM32CubeMX** pour initialiser le SPI4. Cependant, cela a conduit à des **erreurs importantes dans le traitement des données** :

- Les valeurs retournées par le CAN étaient **incohérentes** et semblaient aléatoires.
- Il y avait des problèmes de **synchronisation** entre l'horloge SPI et le signal du CAN.

Certains bits de la donnée numérique étaient **perdus ou mal interprétés**.

Analyse du Problème

Après plusieurs tests, j'ai constaté que la configuration automatique de STM32CubeMX **ne correspondait pas exactement aux besoins du MCP3201**. En effet :

- Le **mode SPI** sélectionné n'était pas le bon. Le MCP3201 fonctionne en **SPI Mode 0** (CPOL = 0, CPHA = 0), ce qui n'était pas correctement défini.
- Le **débit de communication (baud rate prescaler)** était trop élevé, ce qui perturbait la conversion des valeurs.
- L'ordre des bits envoyés et reçus **ne correspondait pas à la spécification du MCP3201**.

Solution Apportée

On a donc dû **configurer manuellement le SPI4** dans STM32CubeMX en :

Modifiant le **mode SPI** pour correspondre aux spécifications du MCP3201 (**Mode 0 : CPOL = 0, CPHA = 0**).
Ajustant la **fréquence de l'horloge SPI** pour qu'elle soit compatible avec la vitesse de conversion du MCP3201 directement sur le code.
Vérifiant que les **données étaient bien lues sur 12 bits**, avec les bits significatifs au bon endroit.

Ces corrections ont permis d'obtenir **des mesures précises et fiables**, sans erreurs de traitement.

Optimisation et Stabilité de l'Affichage

Nous avons rencontré plusieurs problèmes d'affichage, notamment des scintillements et des rafraîchissements inutiles. Voici comment nous les avons corrigés.

- Problème : À chaque mise à jour, l'écran entier était rafraîchi, provoquant un effet de "flash".
- Solution : Mise à jour uniquement des valeurs qui changent : Nous avons utilisé une fonction last_measuring_state() pour vérifier si l'état a changé avant de redessiner le bouton
- Résultat : Plus fluide et plus rapide

Nous avons aussi ajusté la fréquence de mise à jour pour éviter les bugs et ajusté la position des textes et du bouton tactile pour un meilleur équilibre visuel.

Grâce à ces optimisations, nous avons conçu une interface stable, fluide et intuitive. L'écran affiche clairement les mesures, avec un bouton tactile fonctionnel, une barre de progression précise et une réactivité optimisée.

Conclusion

En conclusion, notre projet académique pour la SAE3 en BUT2 GEII nous a permis de tenter de faire communiquer un système électronique capable de mesurer une tension électrique, de la traiter et de l'afficher de manière claire et intuitive. Ce dispositif, centré sur l'utilisation de composants tels que le MCP3201 et la carte STM32, a mis en valeur notre capacité à transformer des mesures techniques en informations accessibles et compréhensibles pour tous les utilisateurs.

Nous avons réussi à approfondir nos connaissances sur la communication SPI (Serial Peripheral Interface), ce qui nous a permis d'assurer une transmission rapide et fiable des données entre le MCP3201 et le microcontrôleur. Cette maîtrise de la communication SPI a renforcé notre compréhension des protocoles de communication et a été essentielle pour le bon fonctionnement de notre système.

L'objectif de ce projet était de se concentrer sur le traitement et l'affichage des données via une carte STM32, en garantissant une expérience utilisateur fluide et précise. Nous avons

travaillé sur la transformation d'une mesure électrique souvent abstraite en une information claire et accessible.

Ce projet nous a offert une opportunité précieuse de mettre en pratique et de renforcer nos compétences en électronique et en programmation, tout en nous sensibilisant à l'importance de l'ergonomie et de la simplicité d'utilisation.